claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf_ae8c9b0f-dd7\agent-a93430bbee80a1c87.jsonl`
- SHA-256: `33227662df45d389fd7a91f9867fa102f97c79824c2a191fd02caa23396e6464`
- Source modified: `2026-06-22T16:38:45+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_ae8c9b0f-dd7`
- session_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`

Transcript

1. user (2026-06-22T16:35:38.411Z)

Review the UNCOMMITTED M12 change in the git worktree at C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.claude\\worktrees\\m11-flywheel (run git -C the-worktree diff + read the new files backend/storage/gdrive_api.py and tests/test_storage_gdrive_api.py). WHAT IT IS: a new gdrive-api:// StorageBackend (GDriveApiStorage) ? the headless Drive-API round-trip (read source + write reel back via the Drive API), distinct from the mount-only gdrive:// (GDriveStorage, which MUST stay untouched). Path-keyed: gdrive-api://<path> resolves each path component to a Drive file id via files().list. All Google libs are lazy-imported; a service is injectable for tests. NO DB migration. Wiring: ref.py (_KNOWN tuple + the path-keyed scheme branch), backend/storage/__init__.py (_backend_class + a hyphen-to-underscore settings lookup in get_storage), backend/config/models.py (StorageConfig.backend/input_backend Literals + a gdrive_api settings dict). Read for grounding: backend/storage/gdrive_api.py, backend/storage/ref.py (parse_uri + StorageRef), backend/storage/__init__.py (_backend_class + get_storage), backend/storage/base.py (the ABC), backend/storage/gcs.py (the DI template it mirrors), backend/storage/gdrive.py (the mount backend ? confirm untouched), backend/config/models.py (StorageConfig). Be adversarial; default to a finding if uncertain.

LENS = STORAGE-LAYER WIRING + NO-REGRESSION + LAZY-IMPORT. Verify:
1. ref.py parse_uri: does gdrive-api://a/b parse to (backend=gdrive-api, bucket=empty, key=a/b)? Confirm _SCHEME_RE allows the hyphen and that adding gdrive-api to the path-keyed branch does NOT change parsing of local/gdrive/gcs/s3/gs. Does an unknown scheme still raise ValueError?
2. get_storage hyphen-to-underscore lookup (cfg.get(name) or cfg.get(name.replace dash to underscore)): confirm hyphen-free names (local/s3/gcs/gdrive) are UNAFFECTED (first lookup wins, byte-identical). EDGE CASE: if someone sets storage.gdrive_api = {} explicitly, does {} being falsy wrongly fall through? (minor) And does an empty-dict settings change any other backend?
3. models.py: the two Literal extensions + the new gdrive_api dict field ? break any existing config test or the loader? Does model_dump() include gdrive_api (used by _storage_dict)? Default {} is inert?
4. _backend_class: the new branch + the updated error message; gdrive_api imported lazily only when name==gdrive-api. CONFIRM that importing backend.storage does NOT import googleapiclient or google.auth ? verify the only google-import sites are _build_service + _media_body, both method-local.
5. gdrive.py (the mount backend): confirm it is UNTOUCHED by the diff.
6. _build_service: with libs absent it raises a clear RuntimeError (owner-actionable), not a bare ImportError. No secrets read from config (only root_folder_id). Confirm.
Return findings + the no_regression verdict with file:line.

2. assistant (2026-06-22T16:35:41.046Z)

I'll analyze this uncommitted M12 change systematically, following the lens of storage-layer wiring, no-regression, and lazy-import verification.

Let me start by examining the git diff and the new/modified files.

3. user (2026-06-22T16:35:41.722Z)

```
 backend/config/models.py                 |  7 +++++--
 backend/storage/__init__.py              | 10 ++++++++--
 backend/storage/ref.py                   |  7 ++++---
 docs/COMPUTE_DECOUPLED_SERVING/README.md |  3 ++-
 4 files changed, 19 insertions(+), 8 deletions(-)
```

4. assistant (2026-06-22T16:35:43.627Z)

Now let me see the full diff to understand what's being changed:

5. user (2026-06-22T16:35:44.096Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index 9c8cf77..015c6a0 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -315,7 +315,7 @@ class StorageConfig(BaseModel):
     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to the
constructor ?
     **secrets are never stored here**, only endpoints / regions / file paths / source-selectors
     (boto3/google auth chains supply credentials). See ``backend/storage/``."""
-    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
+    backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
     # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
     output_root: str = ""
     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
@@ -326,7 +326,7 @@ class StorageConfig(BaseModel):
     # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the input
     # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs are
fetched to a
     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
-    input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
+    input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
     input_root: str = ""
     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
@@ -343,6 +343,9 @@ class StorageConfig(BaseModel):
     s3: dict[str, Any] = Field(default_factory=dict)
     gcs: dict[str, Any] = Field(default_factory=dict)
     gdrive: dict[str, Any] = Field(default_factory=dict)
+    # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the hyphen?underscore
lookup in
+    # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive scope.
+    gdrive_api: dict[str, Any] = Field(default_factory=dict)


 class DeploymentConfig(BaseModel):
diff --git a/backend/storage/__init__.py b/backend/storage/__init__.py
index 6aaf278..fbd2206 100644
--- a/backend/storage/__init__.py
+++ b/backend/storage/__init__.py
@@ -88,7 +88,10 @@ def _backend_class(name: str):
     if name == "gdrive":
         from backend.storage.gdrive import GDriveStorage
         return GDriveStorage
-    raise ValueError(f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive)")
+    if name == "gdrive-api":
+        from backend.storage.gdrive_api import GDriveApiStorage
+        return GDriveApiStorage
+    raise ValueError(f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-
api)")
```

```
 def get_storage(backend: Optional[str] = None, *,
@@ -98,7 +101,10 @@ def get_storage(backend: Optional[str] = None, *,
     ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
     cfg = config or {}
     name = backend or cfg.get("backend", "local")
-    settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
+    # Per-backend settings live under the backend name; tolerate the hyphen?underscore form so
a
+    # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a hyphen
+    # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
+    settings: Dict[str, Any] = dict(cfg.get(name) or cfg.get(name.replace("-", "_")) or {})
     settings.update(overrides)
     return _backend_class(name)(**settings)

diff --git a/backend/storage/ref.py b/backend/storage/ref.py
index 60e2430..f40a63a 100644
--- a/backend/storage/ref.py
+++ b/backend/storage/ref.py
@@ -19,7 +19,7 @@ from datetime import datetime
 from typing import Optional

 _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
-_KNOWN = ("local", "s3", "gcs", "gdrive")
+_KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")


 @dataclass(frozen=True)
@@ -59,8 +59,9 @@ def parse_uri(uri: str) -> StorageRef:
     rest = s[m.end():]
     if scheme == "gs":
         scheme = "gcs"
-    if scheme in ("local", "gdrive"):
-        # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My Drive".
+    if scheme in ("local", "gdrive", "gdrive-api"):
+        # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path under a
Drive-API
+        # root folder (gdrive-api resolves the path ? a Drive file id).
         return StorageRef(scheme, "", rest, s)
     if scheme in _KNOWN:
         bucket, _, key = rest.partition("/")
diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md b/docs/COMPUTE_DECOUPLED_SERVING/README.md
index 56e3084..fb754ad 100644
--- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
+++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
@@ -124,7 +124,7 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate
wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; **+
exposure-safety boot posture** in `startup.py` ? edge-auth-on requires the `allowed_video_root`
jail + CF team/AUD or the server refuses to start, see changelog; consent cols + ToS/DPA ?
owner/counsel) |
 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner
approval | ? | ? |
 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload +
Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval**
owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden
TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model
climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). |
M7,M9 | ? owner (consent/privacy) | ? | **skeleton landed** (see changelog):
`backend/flywheel.py` + `FlywheelConfig` (default-OFF) + gated `_maybe_run_flywheel` hook,
**consent faked-deny**; live activation ? owner/counsel (consent schema) + owned-model/golden
data |
-| **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source
from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user
```

OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
+| **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | **backend landed** (see changelog): `backend/storage/gdrive_api.py` `GDriveApiStorage` + `gdrive-api://` scheme + ADC build; hermetic fake-Drive tests. **Live per-user OAuth app ? owner** |

 Testing strategy is a first-class deliverable ? [`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ? [`runlogs/`](runlogs/).

@@ -169,3 +169,4 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
 - 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (`_enforce_startup_or_exit` ? exit 1) + the worker (`cmd_infer`/`cmd_train` ? rc 2), default-OFF (probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **L4 profile-parity test** (`tests/test_profile_parity.py`): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (`wf_34671ac1`). Suite green, ruff+mypy clean. **? M5 ? ? the real-GPU truly-local runlog (L5) is owner-side** (template pre-staged in `runlogs/`).
 - 2026-06-21 ? **M9 hardening: exposure-safety boot posture** (alpha-shareable "tunnel-to-a-box" enabler): extended `backend/config/startup.py` with a **server-only exposure posture** (new `include_exposure=True` from `_enforce_startup_or_exit`; the worker stays unaffected ? it reads `--video-uri`, not untrusted tenant paths). Gated on `security.edge_auth_enabled` (**default-OFF ? strict no-op**, byte-identical to today): when the owner flips edge auth ON to expose the engine behind the CF-Access gate, three checks fire at **boot** ? `EXPOSED_NO_VIDEO_ROOT` (**fatal**: `allowed_video_root` jail unset ? a tenant could read an arbitrary local file), `EXPOSED_EDGE_AUTH_INCOMPLETE` (**fatal**: CF team-domain/AUD missing ? lifts the current first-request 401 to boot), `EXPOSED_AUTH_EXTRA_MISSING` (**warn**: `PyJWT[crypto]` not importable ? would 500 per request). So a half-configured exposure fails fast instead of leaking/500-ing. +15 unit tests (default-OFF no-op, worker-ignores, both-fatals-compose-with-profile-checks, server-exit, WARN-only-no-exit). Adversarial 3-lens review (`wf_35b735a9`) confirmed default-OFF byte-identical + no bypass; folded a **whitespace-drift** (aligned `auth.resolve_tenant` to `.strip()` team/AUD so "configured" means the same at boot + at request time) + named the config fields in the edge-auth-incomplete message. Pairs with the appliance **T3** network posture (raw `:8000` refused ? owner-run). Suite green, ruff+mypy clean. **? backstops M9 ? the actual exposure flip + CF team/AUD/ingress + key rotation stay owner-side. (Deferred to the PR-2 runbook: document that `upload_dir` must sit under `allowed_video_root`, and that you MUST set `edge_auth_enabled=True` when exposing ? these checks only fire then.)**
 - 2026-06-22 ? **M11 skeleton landed** (data-flywheel harness, D12): `backend/flywheel.py` (`capture_job` ? per-video `VideoWorkspace` via `backend.storage.put`; `shadow_eval` ? `experiment.compare`/`compare_unlabeled`; `propose_promotion` ? `promotion.promote_if_served_safe` with Gemini=control/owned=proposal; `run_for_job` orchestrator) + `FlywheelConfig` (**default-OFF**) + a gated `_maybe_run_flywheel` job-completion hook mirroring M8's cost hook. **TWO independent guards keep it INERT in production:** `flywheel.enabled=False` (default) AND `consent_attested` is a **FAKED hard-deny** (returns False for every job) ? so even enabled it captures **nothing** until the real consent schema lands. This is the PLUMBING only; it reuses the real workspace/eval/promotion/storage code so it works the day consent + the owned model + golden labels are wired. **NO DB migration** (the skeleton persists no consent). +13 tests (default-OFF no-op, enabled-but-consent-denied no-op, consented-capture writes the manifest, delegation of shadow-eval/promote, the hook is best-effort). Suite green, ruff+mypy clean. **? M11 ? ? LIVE activation is owner/counsel (the consent ToS/DPA + schema ? the consent columns) + owner/GPU (the owned model + golden corpus for shadow-eval/promotion). ? the future M9-consent migration is now `v9` (v7 = `locale`, v8 = the compute-picker `compute` column ? the schema ladder drifts as siblings land; check `PRAGMA user_version` before authoring).**
+- 2026-06-22 ? **M12 backend landed** (`gdrive-api://` OAuth/API StorageBackend, D14 Northstar): `backend/storage/gdrive_api.py` `GDriveApiStorage(StorageBackend)` ? the headless **Drive-API** round-trip (read source + write reel back), distinct from the mount-only `gdrive://` (left untouched). Path-keyed `gdrive-api://<path>` resolves each component ? a Drive file id via `files().list`; implements `fetch_to_local` (`get_media().execute()` bytes), `put`

(folder-creating `files().create` + a `_media_body` seam), `exists`/`stat`/`list`/`delete`. Wired the scheme into `ref.py` (`_KNOWN` + the path-keyed branch), `__init__._backend_class`, and the `StorageConfig.backend`/`input_backend` Literals + a `gdrive_api` settings dict (read via a **hyphen?underscore** lookup in `get_storage`). **All Google libs lazy-imported** (`import backend.storage` never pulls them); a `service` is **injectable** ? fully-offline tests via an in-memory fake Drive (round-trip + scheme + registry + the underscore-settings lookup); `_build_service` (ADC over the Drive scope) raises a clear `storage-gdrive-api`-extra error when libs/creds are absent. **NO DB migration.** Adversarial review (`wf_<m12>`). Suite green (1466), ruff+mypy clean. **? M12 ? ? the LIVE per-user interactive OAuth app (consumer "connect your Drive": client id/secret + refresh tokens) is owner-gated; the backend ships the service-account/ADC build + the fake. Reads no secrets from config (creds via ADC only).**

6. assistant (2026-06-22T16:35:48.419Z)

Now let me check the new files and verify the key imports and implementation:

7. user (2026-06-22T16:35:49.296Z)

```
1       """Google Drive **API** backend (OAuth / service-account) ? ``gdrive-api://`` (M12, D14
Northstar).
2
3       The cloud-native Drive round-trip: read a source from + write the stitched reel **back
to** a user's
4       Google Drive **via the Drive API** ? no local mount required (unlike the ``gdrive://``
backend, which
5       is plain local-FS over a *mounted* Drive for Desktop and stays untouched). This is the
headless path a
6       Cloud Run / GPU box uses.
7
8       ``gdrive-api://<path>`` names a path **relative to a root folder** (default the Drive
``root``, or
9       ``storage.gdrive-api.root_folder_id``); the backend resolves each path component to a
Drive file id via
10      ``files().list``. The Google libraries (``google-api-python-client`` + ``google-auth``)
are imported
11      **lazily** ? ``import backend.storage`` never pulls them ? and a ``service`` may be
**injected** for
12      fully-offline tests (mirroring ``GCSStorage(client=...)``).
13
14      ? **Credentials are never in config.** The real service is built from Application
Default Credentials
15      (a service account / workload identity) over the ``drive`` scope. **Per-user interactive
OAuth (the
16      consumer "connect your Drive" flow: client id/secret + refresh tokens) is OWNER-gated**
? this backend
17      ships the service-account/ADC build + the injected fake; the live per-user OAuth app is
a follow-up.
18      """
19      from __future__ import annotations
20
21      import io
22      import os
23      from datetime import datetime
24      from typing import Any, Iterator, Optional
25
26      from backend.storage.base import StorageBackend
27      from backend.storage.ref import StorageRef, StorageStat
28
29      _FOLDER_MIME = "application/vnd.google-apps.folder"
30      _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
31
32
33      def _q_escape(name: str) -> str:
34          """Escape a value for a Drive ``q`` string literal (single-quoted): ``\\`` then
``'``."""
```

```
35          return name.replace("\\", "\\\\").replace("'", "\\'")
36
37
38      class GDriveApiStorage(StorageBackend):
39          """Drive-API-backed storage. Construct with an injected ``service`` (tests) or let
it build one
40          from ADC (production). ``root_folder_id`` anchors relative ``gdrive-api://``
paths."""
41
42          def __init__(self, *, service: Any = None, root_folder_id: str = "root", **_ignored)
-> None:
43              self._root_id = root_folder_id or "root"
44              if service is not None:
45                  self._service = service
46                  return
47              self._service = self._build_service()
48
49          # -- construction (real path; lazy Google imports)
---------------------------------------
50          def _build_service(self) -> Any:
51              try:
52                  import google.auth  # type: ignore
53                  from googleapiclient.discovery import build  # type: ignore
54              except ImportError as e:  # pragma: no cover - exercised only without the extra
installed
55                  raise RuntimeError(
56                      "the gdrive-api backend needs the 'storage-gdrive-api' extra "
57                      "(google-api-python-client + google-auth). Install it on a host that
uses "
58                      "gdrive-api:// URIs; credentials come from ADC (a service account /
workload "
59                      "identity over the Drive scope) ? never from config."
60                  ) from e
61              creds, _ = google.auth.default(scopes=[_DRIVE_SCOPE])
62              return build("drive", "v3", credentials=creds, cache_discovery=False)
63
64          def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
65              """The upload media object for ``files().create`` ? lazy ``MediaFileUpload``
(real path).
66              A test seam: monkeypatch this to avoid the ``googleapiclient`` dependency."""
67              from googleapiclient.http import MediaFileUpload  # type: ignore  # pragma: no
cover
68
69              return MediaFileUpload(local_path, mimetype=content_type, resumable=True)  #
pragma: no cover
70
71          # -- path -> id resolution
------------------------------------------------------------
72          def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
73              """Walk ``key``'s path components from the root folder, returning the leaf's
file id (or
74              ``None`` if a component is missing). With ``create_dirs`` EVERY missing
component is created as
75              a folder ? ``put`` uses this on the parent-directory path (where all components
are folders),
76              so a write into a fresh subfolder works. The read paths leave it ``False`` (a
missing component
77              ? ``None``), never creating anything."""
78              parent = self._root_id
79              parts = [p for p in key.split("/") if p]
80              for part in parts:
81                  q = (f"name = '{_q_escape(part)}' and '{_q_escape(parent)}' in parents "
82                       f"and trashed = false")
83                  found = self._service.files().list(
84                      q=q, spaces="drive", fields="files(id,
```

```
mimeType)").execute().get("files", [])
                if found:
                    parent = found[0]["id"]
                elif create_dirs:
                    parent = self._service.files().create(
                        body={"name": part, "mimeType": _FOLDER_MIME, "parents": [parent]},
                        fields="id").execute()["id"]
                else:
                    return None
            return parent

        def _split(self, key: str) -> tuple[str, str]:
            """Split a ``gdrive-api://`` key into (parent dir path, leaf name)."""
            clean = key.strip("/")
            head, _, name = clean.rpartition("/")
            return head, name

        # -- StorageBackend interface
------------------------------------------------------------
        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError("GDriveApiStorage.fetch_to_local requires a dst path (no
local mount)")
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            file_id = self._resolve_id(ref.key)
            if not file_id:
                raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
            data = self._service.files().get_media(fileId=file_id).execute()  # alt=media ?
raw bytes
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            with io.open(dst, "wb") as fh:
                fh.write(data)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            parent_path, name = self._split(ref.key)
            parent_id = self._resolve_id(parent_path, create_dirs=True) if parent_path else
self._root_id
            if parent_id is None:
                raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
            self._service.files().create(
                body={"name": name, "parents": [parent_id]},
                media_body=self._media_body(str(local_path), content_type),
                fields="id").execute()
            return ref

        def exists(self, ref) -> bool:
            return self._resolve_id(ref.key) is not None

        def stat(self, ref) -> StorageStat:
            file_id = self._resolve_id(ref.key)
            if not file_id:
                raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
            meta = self._service.files().get(
                fileId=file_id, fields="size, modifiedTime, md5Checksum,
mimeType").execute()
            modified = None
            if meta.get("modifiedTime"):
                # Drive returns RFC-3339 (e.g. "2026-06-22T10:00:00.000Z"); normalize the
trailing Z.
                modified = datetime.fromisoformat(meta["modifiedTime"].replace("Z",
"+00:00"))
            return StorageStat(
```

```
142                 size=int(meta.get("size") or 0), modified=modified,
143                 etag=meta.get("md5Checksum"), content_type=meta.get("mimeType"))
144
145         def list(self, prefix) -> Iterator[StorageRef]:
146             folder_id = self._resolve_id(prefix.key) if prefix.key.strip("/") else
self._root_id
147             if not folder_id:
148                 return
149             base = prefix.key.strip("/")
150             page_token: Optional[str] = None
151             while True:
152                 resp = self._service.files().list(
153                     q=f"'{_q_escape(folder_id)}' in parents and trashed = false",
154                     spaces="drive", fields="nextPageToken, files(id, name)",
155                     pageToken=page_token).execute()
156                 for f in resp.get("files", []):
157                     key = f"{base}/{f['name']}" if base else f["name"]
158                     yield StorageRef("gdrive-api", "", key, f"gdrive-api://{key}")
159                 page_token = resp.get("nextPageToken")
160                 if not page_token:
161                     break
162
163         def delete(self, ref) -> None:
164             file_id = self._resolve_id(ref.key)
165             if file_id:
166                 self._service.files().delete(fileId=file_id).execute()
167
```

8. user (2026-06-22T16:35:49.755Z)

```
1       """M12 ? the ``gdrive-api://`` OAuth/API StorageBackend (D14 Northstar).
2
3       Fully offline: a tiny in-memory ``_FakeDrive`` stands in for the googleapiclient Drive
v3 ``service``
4       (injected, mirroring the GCS fake-client tests). No ``google-api-python-client`` /
network needed.
5       Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, and the
full
6       path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete).
7       """
8       import re
9
10      import pytest
11
12      from backend.config import AppConfig
13      from backend.storage import get_storage
14      from backend.storage.gdrive_api import GDriveApiStorage
15      from backend.storage.ref import parse_uri
16
17
18      # ---------------------------------------------------------------- the in-memory fake
Drive service
19
20      class _Exec:
21          def __init__(self, value):
22              self._value = value
23
24          def execute(self):
25              return self._value
26
27
28      class _FakeFiles:
29          def __init__(self, drive):
30              self.d = drive
31
```

```
        def list(self, q="", spaces=None, fields=None, pageToken=None):
            name_m = re.search(r"name = '([^']*)'", q)
            parent_m = re.search(r"'([^']*)' in parents", q)
            parent = parent_m.group(1) if parent_m else None
            out = []
            for nid, n in self.d.nodes.items():
                if parent is not None and parent not in n["parents"]:
                    continue
                if name_m and n["name"] != name_m.group(1):
                    continue
                if nid == "root":
                    continue
                out.append({"id": nid, "name": n["name"], "mimeType": n["mimeType"]})
            return _Exec({"files": out})  # one page, no nextPageToken

        def create(self, body=None, media_body=None, fields=None):
            nid = self.d.new_id()
            self.d.nodes[nid] = {
                "name": body["name"], "mimeType": body.get("mimeType", "application/octet-
stream"),
                "parents": list(body.get("parents", [])),
            }
            if media_body is not None:  # in tests _media_body is patched to return the
local path
                with open(media_body, "rb") as fh:
                    self.d.content[nid] = fh.read()
            return _Exec({"id": nid})

        def get_media(self, fileId=None):
            return _Exec(self.d.content.get(fileId, b""))

        def get(self, fileId=None, fields=None):
            n = self.d.nodes[fileId]
            data = self.d.content.get(fileId, b"")
            return _Exec({"size": str(len(data)), "modifiedTime":
"2026-06-22T10:00:00.000Z",
                          "md5Checksum": "deadbeef", "mimeType": n["mimeType"]})

        def delete(self, fileId=None):
            self.d.nodes.pop(fileId, None)
            self.d.content.pop(fileId, None)
            return _Exec({})


    class _FakeDrive:
        def __init__(self):
            self.nodes = {"root": {"name": "root", "mimeType": "folder", "parents": []}}
            self.content = {}
            self._n = 0

        def new_id(self):
            self._n += 1
            return f"id{self._n}"

        def files(self):
            return _FakeFiles(self)


    @pytest.fixture
    def be(monkeypatch):
        """A GDriveApiStorage on a fake service, with _media_body patched to the local path
(no
        googleapiclient dependency ? the fake `create` reads the bytes from that path)."""
        monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
        return GDriveApiStorage(service=_FakeDrive())
```

```
93
94
95      # ------------------------------------------------------------------------ scheme +
registry wiring
96
97      def test_parse_uri_gdrive_api_is_path_keyed():
98          ref = parse_uri("gdrive-api://Coaching/2026/clip.mp4")
99          assert (ref.backend, ref.bucket, ref.key) == ("gdrive-api", "",
"Coaching/2026/clip.mp4")
100         assert ref.uri == "gdrive-api://Coaching/2026/clip.mp4"
101
102
103     def test_unknown_scheme_still_rejected():
104         with pytest.raises(ValueError):
105             parse_uri("gdrive-zzz://x")
106
107
108     def test_config_accepts_gdrive_api_backend():
109         cfg = AppConfig.model_validate({"storage": {"backend": "gdrive-api",
"input_backend": "gdrive-api"}})
110         assert cfg.storage.backend == "gdrive-api"
111         assert cfg.storage.input_backend == "gdrive-api"
112
113
114     def test_get_storage_wires_gdrive_api_and_reads_underscore_settings(monkeypatch):
115         """Registry resolves 'gdrive-api' ? GDriveApiStorage, and its settings come from the
116         `gdrive_api` field (hyphen?underscore lookup) merged with overrides (here, the
injected service)."""
117         monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
118         cfg = {"backend": "gdrive-api", "gdrive_api": {"root_folder_id": "FOLDER42"}}
119         be = get_storage("gdrive-api", config=cfg, service=_FakeDrive())
120         assert isinstance(be, GDriveApiStorage)
121         assert be._root_id == "FOLDER42"
122
123
124     # ------------------------------------------------------------------------------- full
round-trip
125
126     def test_roundtrip_put_exists_stat_fetch_list_delete(be, tmp_path):
127         src = tmp_path / "clip.mp4"
128         src.write_bytes(b"VIDEO-BYTES")
129         ref = parse_uri("gdrive-api://Coaching/2026/clip.mp4")
130
131         be.put(str(src), ref)                       # creates the Coaching/ + 2026/ folders
+ the file
132         assert be.exists(ref)
133         assert be.stat(ref).size == len(b"VIDEO-BYTES")
134         assert be.stat(ref).etag == "deadbeef"
135
136         dst = tmp_path / "out" / "clip.mp4"
137         be.fetch_to_local(ref, str(dst))
138         assert dst.read_bytes() == b"VIDEO-BYTES"
139
140         listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching/2026"))]
141         assert listed == ["gdrive-api://Coaching/2026/clip.mp4"]
142
143         be.delete(ref)
144         assert not be.exists(ref)
145
146
147     def test_fetch_missing_raises(be, tmp_path):
148         with pytest.raises(FileNotFoundError):
149             be.fetch_to_local(parse_uri("gdrive-api://nope/missing.mp4"), str(tmp_path /
"x"))
150
```

```
151
152     def test_fetch_requires_dst(be):
153         with pytest.raises(ValueError):
154             be.fetch_to_local(parse_uri("gdrive-api://a/b.mp4"), None)
155
156
157     def test_put_at_root_uses_root_folder(be, tmp_path):
158         src = tmp_path / "top.json"
159         src.write_text("{}")
160         ref = parse_uri("gdrive-api://top.json")
161         be.put(str(src), ref)
162         assert be.exists(ref)
163
164
165     def test_build_service_without_libs_raises_actionable_error(monkeypatch):
166         """No injected service + the Google libs absent ? a clear, owner-actionable error
(not ImportError
167         leaking through). googleapiclient is not installed in the test env, so this
exercises the real path."""
168         with pytest.raises(RuntimeError, match="storage-gdrive-api"):
169             GDriveApiStorage()
170
```

9. assistant (2026-06-22T16:35:53.038Z)

Now let me verify the other files mentioned in the requirements:

10. user (2026-06-22T16:35:53.881Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
```

```
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53          """
54          s = str(uri)
55          m = _SCHEME_RE.match(s)
56          if not m:
57              return StorageRef("local", "", s, f"local://{s}")
58          scheme = m.group(1).lower()
59          rest = s[m.end():]
60          if scheme == "gs":
61              scheme = "gcs"
62          if scheme in ("local", "gdrive", "gdrive-api"):
63              # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path
under a Drive-API
64              # root folder (gdrive-api resolves the path ? a Drive file id).
65              return StorageRef(scheme, "", rest, s)
66          if scheme in _KNOWN:
67              bucket, _, key = rest.partition("/")
68              return StorageRef(scheme, bucket, key, s)
69          raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
70
71
72      def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
73          """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
74
75          Precedence (first match wins):
76            1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
77            2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
78            3. A bare/relative name + a non-empty ``input_root`` ? joined under it
79               (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
80            4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
81               ``"gcs://bucket/x"``).
82            5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
83
84          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
85          path used today, preserving byte-for-byte behaviour.
86          """
87          s = str(raw)
```

```
88          if _SCHEME_RE.match(s):           # 1. explicit scheme wins
89              return s
90          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
91          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
92          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
93          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
94              return s
95          if input_root:                    # 3. join a relative name under the configured
root
96              return input_root.rstrip("/") + "/" + s.lstrip("/")
97          if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the
declared backend
98              return f"{input_backend}://{s}"
99          return s                          # 5. bare/relative local path ? unchanged
100
```

11. user (2026-06-22T16:35:54.175Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13      from __future__ import annotations
14
15      import os
16      import tempfile
17      from typing import Any, Dict, Iterator, Optional
18
19      from backend.storage.base import StorageBackend
20      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22      __all__ = [
23          "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24          "get_storage", "fetch", "put", "list_uris",
25          "acquire_local_input", "input_is_local", "resolve_input_ref",
26      ]
27
28
29      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31          if storage_config is None:
32              return {}
33          if hasattr(storage_config, "model_dump"):
34              return storage_config.model_dump()  # type: ignore[no-any-return]
35          return dict(storage_config)
36
37
38      def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39          """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
```

```
        config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
        callers at the API/CLI boundary catch that and return a clean client error (never a
500).

        Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
        ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
        sc = _storage_dict(storage_config)
        uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
                                input_root=sc.get("input_root", ""))
        return parse_uri(uri)


    def input_is_local(raw: str, storage_config: Any = None) -> bool:
        """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
        plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
        that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
        ``resolve_input_ref``)."""
        return resolve_input_ref(raw, storage_config).backend == "local"


    def acquire_local_input(raw: str, storage_config: Any = None, *,
                            scratch_parent: "str | None" = None) -> str:
        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.

        Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
        byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
        non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
        path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
        regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
        worker's ``cmd_infer`` already ships."""
        sc = _storage_dict(storage_config)
        ref = resolve_input_ref(raw, sc)
        if ref.backend == "local":
            return ref.key  # identity ? no temp, no copy, byte-identical to passing the
path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage
            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage
            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage
            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage
            return GDriveStorage
        if name == "gdrive-api":
```

```
92              from backend.storage.gdrive_api import GDriveApiStorage
93              return GDriveApiStorage
94          raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive|gdrive-api)")
95
96
97      def get_storage(backend: Optional[str] = None, *,
98                      config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
99          """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
100         the implementation; its per-backend settings come from ``config[backend]`` merged
with
101         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
102         cfg = config or {}
103         name = backend or cfg.get("backend", "local")
104         # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
105         # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
106         # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
107         settings: Dict[str, Any] = dict(cfg.get(name) or cfg.get(name.replace("-", "_")) or
{})
108         settings.update(overrides)
109         return _backend_class(name)(**settings)
110
111
112     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
113               config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
114         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
115         (no copy); for s3/gcs it downloads to ``dst``."""
116         ref = parse_uri(uri)
117         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
118
119
120     def put(local_path: "str | os.PathLike[str]", uri: str, *,
121             config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
122         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
123         ref = parse_uri(uri)
124         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
125         return uri
126
127
128     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
129         """Yield the URI of every object under ``prefix_uri``."""
130         ref = parse_uri(prefix_uri)
131         for r in get_storage(ref.backend, config=config).list(ref):
132             yield r.uri
133
```

12. user (2026-06-22T16:35:54.712Z)

```
1       """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3       Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4       live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5       pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
```

```
6       no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7       """
8       from __future__ import annotations
9
10      import os
11      from abc import ABC, abstractmethod
12      from typing import Iterator
13
14      from backend.storage.ref import StorageRef, StorageStat
15
16
17      class StorageBackend(ABC):
18          """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19          worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21          @abstractmethod
22          def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                             *, overwrite: bool = False) -> str:
24              """Make the object available as a local file; return its local path. For the
25              ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27          @abstractmethod
28          def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29                  content_type: "str | None" = None) -> StorageRef:
30              """Upload/copy a local file to ``ref``; return ``ref``."""
31
32          @abstractmethod
33          def exists(self, ref: StorageRef) -> bool: ...
34
35          @abstractmethod
36          def stat(self, ref: StorageRef) -> StorageStat: ...
37
38          @abstractmethod
39          def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40              """Yield a ``StorageRef`` per object under ``prefix``."""
41
42          @abstractmethod
43          def delete(self, ref: StorageRef) -> None: ...
44
45          def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46              """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47              sign (``local`` cannot; Drive only emulates) ? callers must treat
48              ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49              raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50
```

13. user (2026-06-22T16:35:55.445Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
```

```
    Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]


    class ValidationRelevanceConfig(BaseModel):
        court_green_lower: list[int] = Field(default=[35, 40, 40])
        court_green_upper: list[int] = Field(default=[85, 255, 255])
        court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


    class ValidationConfig(BaseModel):
        min_resolution_width: int = 1280
        min_resolution_height: int = 720
        min_fps: float = 29.9
        min_blur_threshold: float = 20.0
        max_scene_cuts_per_minute: float = 0.5
        relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)


    class TrackNetConfig(BaseModel):
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""
        queue_length: int = 5
        stage_in_wsl: bool = True
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
        # decoded frame cache) are deleted automatically ? they are pure intermediates,
        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
        # 0 = disabled.
        min_free_gb: float = 0.0


    class WasbIndexConfig(BaseModel):
        """Typed config for the live WASB shuttle substrate (indexing.wasb).

        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
        these knobs actually take effect ? previously this whole block was silently dropped
        because nested config sections defaulted to extra='ignore'.
        """
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
        # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
        # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
        # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
        device: str = "cuda"        # auto | cuda | mps | cpu
```

```
75          python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
76          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79          keep_frames: bool = False
80          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81          min_free_gb: float = 0.0
82          # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83          # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84          # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85          # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86          # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87          # Tri-state:
88          #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89          #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90          #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91          stream_video: Optional[bool] = None
92
93
94      class FusionConfig(BaseModel):
95          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97          Every default reproduces control behaviour: the fusion segmenter persists the
98          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99          true ? the person-cue features, but it does NOT gate the served handoff unless a
100         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102         supplied ? off-court persons (spectators / adjacent courts) are excluded.
103         """
104         # Which trajectory substrate seeds the windows (shuttle stays primary).
105         substrate: Literal["wasb", "tracknet"] = "wasb"
106         # Windowing velocity threshold for the fusion family (the engine reads
107         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109         velocity_threshold: float = 0.02
110         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111         # enabled ? so the default path never imports torch / touches the GPU.
112         cue_flags: dict[str, bool] = Field(default_factory=dict)
113         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115         min_crossings: int = Field(default=0, ge=0)
116         # Served Gate B: when the person cue is on, drop windows with median in-court
players
117         # < this. 0 = OFF.
118         min_players: int = Field(default=0, ge=0)
119         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120         court_polygon: Optional[list[list[float]]] = None
121         # Net line for the person two-sided-motion split (person features only ? the shuttle
```

```
122         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123         net_axis: Literal["x", "y"] = "y"
124         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135         detector_impl: str = "auto"
136         motion_threshold: float = 0.08
137         min_segment_duration: float = 3.0
138         dead_time_merge_gap: float = 2.0
139         chunk_duration: float = 90.0
140         chunk_overlap: float = 10.0
141         detector_model: str = "fasterrcnn_resnet50_fpn"
142         detector_score_threshold: float = 0.5
143         yolo_velocity_threshold: float = 0.15
144         yolo_frame_skip: int = 3
145         yolo_tracker: str = "bytetrack.yaml"
146         yolo_prefetch_workers: int = 2
147         min_action_window_duration: float = 2.0
148         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
149         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
151         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155         max_window_duration: float = 6.0
156         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
157         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160         # Only cuts (recall can't drop). OFF by default until measured/promoted.
161         window_hard_cap: bool = False
162         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
```

```
"R1". ***
169         window_inactivity_end: float = 1.5
170         inactivity_rally_thresh: float = 0.20
171         inactivity_min_density: float = 0.30
172         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180         window_blob_fallback: bool = False
181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
190         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191         # chunks of a high-bitrate (4K) source blow past the provider upload cap
192         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195         # Matches gemini_refine's --clip-scale-width default.
196         ai_chunk_scale_width: int = 1280
197         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202         skip_ai_handoff: bool = False
203         # Optional debug knob: trim every video to the first N seconds before processing.
204         debug_duration: Optional[float] = None
205
206         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208         fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210         @field_validator("default_segmenter")
211         @classmethod
212         def segmenter_must_be_registered(cls, v: str) -> str:
213             # Registry check happens at runtime in main/segmenter selection.
214             # Here we just allow any string for forward compatibility.
215             return v
216
217         @model_validator(mode="after")
```

```
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
        # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
        # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
        # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
        # local detector over-fires and its false positives are mostly SHORT (motion blips,
        # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
        # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
        min_rally_duration: float = Field(default=0.0, ge=0.0)
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
```

```
272         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
273         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302         # application's audience tag. NO secrets here ? these are public identifiers.
303         edge_auth_enabled: bool = False
304         cf_team_domain: str = ""        # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305         cf_access_aud: str = ""         # the CF Access application AUD tag the token must
carry
306         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
308         # OFF ? Cf-Access is a header, not a cookie.)
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
318         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
```

```
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
346         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
347         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
348         gdrive_api: dict[str, Any] = Field(default_factory=dict)
349
350
351     class DeploymentConfig(BaseModel):
352         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
353         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
354
355         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
356         behaves exactly as before this section existed. A profile is opt-in (``config.json``
357         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
358         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
359         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
360         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
```

14. assistant (2026-06-22T16:35:57.690Z)

Now let me verify that gdrive.py (the mount backend) is untouched:

15. user (2026-06-22T16:35:58.263Z)

(Bash completed with no output)

16. assistant (2026-06-22T16:36:00.762Z)

Good - no changes. Now let me verify the scheme regex and the _KNOWN tuple to check for regressions:

17. user (2026-06-22T16:36:04.373Z)

```
gdrive-api match: gdrive-api
local:// match: local
s3:// match: s3
gcs:// match: gcs
gdrive:// match: gdrive
gs:// match: gs
9invalid match: None
```

18. assistant (2026-06-22T16:36:06.673Z)

Perfect! Now let me test the parse_uri logic to ensure all schemes parse correctly and unknowns still raise ValueError:

19. user (2026-06-22T16:36:10.205Z)

```
gdrive-api://a/b/c -> backend='gdrive-api', bucket='', key='a/b/c'
gdrive://x/y -> backend='gdrive', bucket='', key='x/y'
local:///path -> backend='local', bucket='', key='/path'
s3://bucket/key -> backend='s3', bucket='bucket', key='key'
gs://bucket/key -> backend='gcs', bucket='bucket', key='key'
Unknown scheme correctly raised: unsupported storage URI scheme: gdrive-zzz:// (in 'gdrive-
zzz://x')
```

20. assistant (2026-06-22T16:36:12.589Z)

Excellent! Now let me verify the hyphen-to-underscore lookup in get_storage, testing that hyphen-free names are NOT affected:

21. user (2026-06-22T16:36:20.453Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 7, in <module>
    be = get_storage('s3', config=cfg_s3)
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\m11-flywheel\backend\storage\__init__.py", line 109, in get_storage
    return _backend_class(name)(**settings)
           ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\m11-flywheel\backend\storage\s3.py", line 29, in __init__
    import boto3  # type: ignore  # lazy: only when an s3:// URI is actually used
    ^^^^^^^^^^^^
ModuleNotFoundError: No module named 'boto3'
```

22. assistant (2026-06-22T16:36:22.602Z)

That's expected (boto3 not installed). Let me test with local only:

23. user (2026-06-22T16:36:29.699Z)

```
Exit code 1
  File "<string>", line 15
    print(f'Empty local={} lookup result: {settings}')
                          ^
SyntaxError: f-string: valid expression required before '}'
```

24. assistant (2026-06-22T16:36:31.300Z)

Let me fix the f-string:

25. user (2026-06-22T16:36:37.116Z)

```
Local backend type: LocalStorage
First lookup wins for hyphen-free names: OK
Empty local dict lookup result: {}
gdrive-api lookup (hyphen->underscore): {'root_folder_id': 'FOLDER42'}
Hyphenated key lookup: {'hyphen_form': 'yes'}
Underscored key lookup: {'underscore_form': 'yes'}
Precedence test (first lookup wins): True
```

26. assistant (2026-06-22T16:36:39.320Z)

Perfect! Now let me verify that importing backend.storage does NOT import googleapiclient or google.auth:

27. user (2026-06-22T16:36:43.606Z)

```
? No Google libs imported on backend.storage import
backend.storage.ref loaded: True
backend.storage.base loaded: True
backend.storage.local loaded: False
```

28. assistant (2026-06-22T16:36:45.857Z)

Perfect! Now let me verify that _build_service raises the correct error when Google libs are missing:

29. user (2026-06-22T16:36:48.759Z)

```
? Correct error message: the gdrive-api backend needs the 'storage-gdrive-api' extra (google-
api-python-c ...
```

30. assistant (2026-06-22T16:36:51.456Z)

Excellent! Now let me verify that the AppConfig loader accepts the new gdrive-api backend:

31. user (2026-06-22T16:36:55.253Z)

```
Storage backend: gdrive-api
Input backend: gdrive-api
gdrive_api settings: {'root_folder_id': 'FOLDER123'}
model_dump includes gdrive_api: True
gdrive_api in dump: {'root_folder_id': 'FOLDER123'}
```

32. assistant (2026-06-22T16:36:57.255Z)

Perfect! Now let me run the test suite to ensure everything passes:

33. user (2026-06-22T16:36:59.138Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 9 items
```

```
tests/test_storage_gdrive_api.py::test_parse_uri_gdrive_api_is_path_keyed PASSED [ 11%]
tests/test_storage_gdrive_api.py::test_unknown_scheme_still_rejected PASSED [ 22%]
tests/test_storage_gdrive_api.py::test_config_accepts_gdrive_api_backend PASSED [ 33%]
tests/test_storage_gdrive_api.py::test_get_storage_wires_gdrive_api_and_reads_underscore_setting
s PASSED [ 44%]
tests/test_storage_gdrive_api.py::test_roundtrip_put_exists_stat_fetch_list_delete PASSED [ 55%]
tests/test_storage_gdrive_api.py::test_fetch_missing_raises PASSED       [ 66%]
tests/test_storage_gdrive_api.py::test_fetch_requires_dst PASSED         [ 77%]
tests/test_storage_gdrive_api.py::test_put_at_root_uses_root_folder PASSED [ 88%]
tests/test_storage_gdrive_api.py::test_build_service_without_libs_raises_actionable_error PASSED
[100%]

============================== 9 passed in 0.31s ===============================
```

34. assistant (2026-06-22T16:37:01.741Z)

Perfect! All tests pass. Now let me verify that the imports in gdrive_api.py only pull Google libs in the methods where they're used:

35. user (2026-06-22T16:37:02.861Z)

```
50          def _build_service(self) -> Any:
51              try:
52                  import google.auth  # type: ignore
53                  from googleapiclient.discovery import build  # type: ignore
54              except ImportError as e:  # pragma: no cover - exercised only without the extra
installed
55                  raise RuntimeError(
56                      "the gdrive-api backend needs the 'storage-gdrive-api' extra "
57                      "(google-api-python-client + google-auth). Install it on a host that
uses "
58                      "gdrive-api:// URIs; credentials come from ADC (a service account /
workload "
59                      "identity over the Drive scope) ? never from config."
60                  ) from e
61              creds, _ = google.auth.default(scopes=[_DRIVE_SCOPE])
62              return build("drive", "v3", credentials=creds, cache_discovery=False)
63
64          def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
65              """The upload media object for ``files().create`` ? lazy ``MediaFileUpload``
(real path).
66              A test seam: monkeypatch this to avoid the ``googleapiclient`` dependency."""
67              from googleapiclient.http import MediaFileUpload  # type: ignore  # pragma: no
cover
68
69              return MediaFileUpload(local_path, mimetype=content_type, resumable=True)  #
pragma: no cover
70
71          # -- path -> id resolution
-------------------------------------------------------------
72          def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
73              """Walk ``key``'s path components from the root folder, returning the leaf's
file id (or
74              ``None`` if a component is missing). With ``create_dirs`` EVERY missing
component is created as
```

36. assistant (2026-06-22T16:37:05.867Z)

Perfect! Google imports are only in:
1. `_build_service` (method-local, line 50-62)
2. `_media_body` (method-local, line 64-69)

Now let me verify that secrets are NOT read from config by checking what _build_service does:

37. user (2026-06-22T16:37:10.979Z)

```
Potential config reads in _build_service:
  "identity over the Drive scope) ? never from config."
? Uses google.auth.default (ADC) ? credentials from environment/workload, not config
? No secrets read from config
```

38. assistant (2026-06-22T16:37:13.215Z)

Great! That's just in the error message. Now let me do a comprehensive test to make sure no regressions exist by running a broader test suite:

39. user (2026-06-22T16:37:15.058Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... ERROR: file or directory not found: tests/test_storage_ref.py

collected 0 items

============================= no tests ran in 0.03s ============================
```

40. assistant (2026-06-22T16:37:16.657Z)

Let me check what test files exist:

41. user (2026-06-22T16:37:17.561Z)

```
tests/test_ablation.py
tests/test_annotation_providers.py
tests/test_api_cloud_dispatch.py
tests/test_api_endpoints.py
tests/test_app_home.py
tests/test_app_launcher.py
tests/test_async_jobs.py
tests/test_audio_features.py
tests/test_automated_provider.py
tests/test_base.py
tests/test_billing.py
tests/test_bind_and_host.py
tests/test_calibrate_local.py
tests/test_calibrate_wasb.py
tests/test_calibration.py
tests/test_cleanup_caches.py
tests/test_compiler.py
tests/test_compute_backend.py
tests/test_config.py
tests/test_config_deployment.py
```

42. assistant (2026-06-22T16:37:19.132Z)

Let me run the config tests:

43. user (2026-06-22T16:37:23.609Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
```

```
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 21 items

tests/test_config.py::test_load_default_config PASSED                    [  4%]
tests/test_config.py::test_logging_config_default_quiets_http PASSED     [  9%]
tests/test_config.py::test_skip_ai_handoff_default_off PASSED            [ 14%]
tests/test_config.py::test_skip_ai_handoff_file_override PASSED          [ 19%]
tests/test_config.py::test_logging_config_file_override PASSED           [ 23%]
tests/test_config.py::test_fusion_config_defaults_reproduce_control PASSED [ 28%]
tests/test_config.py::test_fusion_config_gate_a_opt_in_via_min_crossings PASSED [ 33%]
tests/test_config.py::test_served_default_segmenter_is_gemini PASSED     [ 38%]
tests/test_config.py::test_fusion_config_file_override PASSED            [ 42%]
tests/test_config.py::test_file_override PASSED                          [ 47%]
tests/test_config.py::test_extra_keys_tolerated PASSED                   [ 52%]
tests/test_config.py::test_unknown_keys_warn_with_dotted_paths PASSED    [ 57%]
tests/test_config.py::test_recognized_config_does_not_warn PASSED        [ 61%]
tests/test_config.py::test_non_object_config_degrades_to_defaults PASSED [ 66%]
tests/test_config.py::test_save_default_config_roundtrip PASSED          [ 71%]
tests/test_config.py::test_model_is_the_source_of_truth PASSED           [ 76%]
tests/test_config.py::test_output_dir_field_default_and_override PASSED  [ 80%]
tests/test_config.py::test_config_shallow_section_replacement PASSED     [ 85%]
tests/test_config.py::test_config_unknown_key_paths_warning_and_tolerate PASSED [ 90%]
tests/test_config.py::test_get_returns_dict_for_nested_sections_legacy_compat PASSED [ 95%]
tests/test_config.py::test_config_roundtrip_and_typed_reaches_pipeline PASSED [100%]

============================== 21 passed in 2.02s ==============================
```

44. assistant (2026-06-22T16:37:25.629Z)

Excellent! Now let me check if there are any storage-related tests:

45. user (2026-06-22T16:37:26.373Z)

```
tests/test_storage.py
tests/test_storage_gdrive_api.py
tests/__pycache__/test_storage.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_storage_gdrive_api.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_storage_gdrive_api.cpython-313.pyc
```

46. assistant (2026-06-22T16:37:27.980Z)

Perfect! Let me run the storage tests:

47. user (2026-06-22T16:37:30.225Z)

```
============================== test session starts ==============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 39 items

tests/test_storage.py::test_parse_uri[/data/x.mp4-local--/data/x.mp4] PASSED [  2%]
tests/test_storage.py::test_parse_uri[rel/x.mp4-local--rel/x.mp4] PASSED [  5%]
tests/test_storage.py::test_parse_uri[C:/Users/a/x.mp4-local--C:/Users/a/x.mp4] PASSED [  7%]
tests/test_storage.py::test_parse_uri[C:\\Users\\a\\x.mp4-local--C:\\Users\\a\\x.mp4] PASSED [
10%]
tests/test_storage.py::test_parse_uri[local:///abs/x.mp4-local--/abs/x.mp4] PASSED [ 12%]
tests/test_storage.py::test_parse_uri[s3://rally/videos/m.mp4-s3-rally-videos/m.mp4] PASSED [
15%]
```

```
tests/test_storage.py::test_parse_uri[gs://rally-eu/w/model.json-gcs-rally-eu-w/model.json]
PASSED [ 17%]
tests/test_storage.py::test_parse_uri[gcs://rally/x-gcs-rally-x] PASSED  [ 20%]
tests/test_storage.py::test_parse_uri[gdrive://Sharing/clip.mp4-gdrive--Sharing/clip.mp4] PASSED
[ 23%]
tests/test_storage.py::test_parse_uri_rejects_unknown_scheme PASSED      [ 25%]
tests/test_storage.py::test_local_fetch_is_identity_passthrough PASSED   [ 28%]
tests/test_storage.py::test_local_put_and_roundtrip PASSED               [ 30%]
tests/test_storage.py::test_local_exists_stat_list_delete PASSED         [ 33%]
tests/test_storage.py::test_local_signed_url_raises PASSED               [ 35%]
tests/test_storage.py::test_list_uris_helper PASSED                      [ 38%]
tests/test_storage.py::test_get_storage_defaults_to_local PASSED         [ 41%]
tests/test_storage.py::test_get_storage_unknown_backend_raises PASSED    [ 43%]
tests/test_storage.py::test_gdrive_not_mounted_raises_with_remedy PASSED [ 46%]
tests/test_storage.py::test_gdrive_mounted_but_unsynced_path_raises_with_remedy PASSED [ 48%]
tests/test_storage.py::test_s3_backend_roundtrip_with_fake_client PASSED [ 51%]
tests/test_storage.py::test_s3_exists_false_on_missing PASSED            [ 53%]
tests/test_storage.py::test_gcs_backend_roundtrip_with_fake_client PASSED [ 56%]
tests/test_storage.py::test_gdrive_backend_roundtrip_over_local_mount PASSED [ 58%]
tests/test_storage.py::test_gdrive_put_writes_into_the_mount PASSED      [ 61%]
tests/test_storage.py::test_gdrive_mount_root_from_env PASSED            [ 64%]
tests/test_storage.py::test_gdrive_autodetect_is_safe_across_platforms PASSED [ 66%]
tests/test_storage.py::test_gdrive_exists_false_on_missing PASSED        [ 69%]
tests/test_storage.py::test_gdrive_signed_url_not_supported PASSED       [ 71%]
tests/test_storage.py::test_importing_storage_does_not_pull_cloud_sdks PASSED [ 74%]
tests/test_storage.py::test_config_storage_defaults_are_non_breaking PASSED [ 76%]
tests/test_storage_gdrive_api.py::test_parse_uri_gdrive_api_is_path_keyed PASSED [ 79%]
tests/test_storage_gdrive_api.py::test_unknown_scheme_still_rejected PASSED [ 82%]
tests/test_storage_gdrive_api.py::test_config_accepts_gdrive_api_backend PASSED [ 84%]
tests/test_storage_gdrive_api.py::test_get_storage_wires_gdrive_api_and_reads_underscore_setting
s PASSED [ 87%]
tests/test_storage_gdrive_api.py::test_roundtrip_put_exists_stat_fetch_list_delete PASSED [ 89%]
tests/test_storage_gdrive_api.py::test_fetch_missing_raises PASSED       [ 92%]
tests/test_storage_gdrive_api.py::test_fetch_requires_dst PASSED         [ 94%]
tests/test_storage_gdrive_api.py::test_put_at_root_uses_root_folder PASSED [ 97%]
tests/test_storage_gdrive_api.py::test_build_service_without_libs_raises_actionable_error PASSED
[100%]

============================== 39 passed in 0.52s ==============================
```

48. assistant (2026-06-22T16:37:32.317Z)

Perfect! All tests pass. Now let me do a final comprehensive check of the implementation against the requirements:

49. user (2026-06-22T16:37:54.325Z)

```
======================================================================
1. SCHEME REGEX + _KNOWN TUPLE CHECK
======================================================================
Regex matches "gdrive-api": True
Captured scheme: gdrive-api
_KNOWN tuple: ('local', 's3', 'gcs', 'gdrive', 'gdrive-api')
gdrive-api in _KNOWN: True

Other scheme parsing (no regression):
  ? local:///path -> local
  ? gdrive://My Drive/clip -> gdrive
  ? s3://bucket/key -> s3
  ? gcs://bucket/key -> gcs
  ? gs://bucket/key -> gcs

Unknown scheme rejection:
  ? Raised ValueError: unsupported storage URI scheme: unknown:// (in 'un...
```

```
======================================================================
2. GET_STORAGE HYPHEN-TO-UNDERSCORE LOOKUP
======================================================================
? Backend "local": {'config': 'local_val'}
? Backend "gdrive-api": {'config': 'underscore_val'}
? Backend "s3": {'endpoint': 'x'}

======================================================================
3. STORAGE CONFIG LITERALS + GDRIVE_API FIELD
======================================================================
StorageConfig fields:
  backend type: typing.Literal['local', 's3', 'gcs', 'gdrive', 'gdrive-api']
  input_backend type: typing.Literal['local', 's3', 'gcs', 'gdrive', 'gdrive-api']
  Has gdrive_api field: True
Config loads gdrive-api: ?
gdrive_api in model_dump: True

======================================================================
4. LAZY GOOGLE IMPORTS (no import on backend.storage import)
======================================================================
? No Google libs imported on backend.storage import

======================================================================
5. _BUILD_SERVICE ERROR HANDLING
======================================================================
? RuntimeError with "storage-gdrive-api" message

======================================================================
6. NO SECRETS FROM CONFIG
======================================================================
Uses google.auth.default (ADC): True
Reads secrets from config: False
? Creds from ADC only, not config

======================================================================
SUMMARY: All checks passed ?
======================================================================
```

50. assistant (2026-06-22T16:38:00.123Z)

Excellent! Let me verify one more thing - that the gdrive.py backend is truly untouched:

51. user (2026-06-22T16:38:01.121Z)

(Bash completed with no output)

52. assistant (2026-06-22T16:38:03.223Z)

Perfect - no changes. Let me create a final comprehensive summary by checking the key files one more time:

53. user (2026-06-22T16:38:04.391Z)

```
318         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
```

```
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
346         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
347         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
348         gdrive_api: dict[str, Any] = Field(default_factory=dict)
349
350
351     class DeploymentConfig(BaseModel):
352         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
```

54. assistant (2026-06-22T16:38:06.668Z)

Perfect! Now let me compile the final verification report:

55. user (2026-06-22T16:38:33.289Z)

Structured output provided successfully

56. assistant (2026-06-22T16:38:45.143Z)

Summary

I have completed a thorough adversarial review of the M12 gdrive-api backend change. Here are the key findings:

Verification Results (All Pass ?)

1. ref.py parse_uri & scheme regex
- _SCHEME_RE correctly allows hyphens (RFC 3986 compliant)
- gdrive-api:// parses to (backend=gdrive-api, bucket="", key=path)
- Path-keyed logic correctly includes gdrive-api alongside local/gdrive
- Unknown schemes still raise ValueError as expected
- No regression to local/s3/gcs/gdrive/gs parsing

2. get_storage hyphen-to-underscore lookup
- Hyphen-free backends (local/s3/gcs/gdrive) use first cfg.get(name) and return immediately ? NO fallthrough
- gdrive-api correctly falls through to gdrive_api field
- Empty dict {} handled safely (the `or` chain continues correctly)
- Lookup logic is transparent to existing backends (byte-identical)

3. models.py Literal extensions + gdrive_api field
- StorageConfig.backend and input_backend Literals updated with "gdrive-api"
- New gdrive_api: dict[str, Any] field defaults to empty dict (safe, non-breaking)
- AppConfig.model_validate accepts gdrive-api config
- model_dump() includes gdrive_api (used by _storage_dict)
- All 21 pre-existing config tests pass

4. Lazy Google imports verified
- `import backend.storage` does NOT import google.auth or googleapiclient
- Google libs only loaded in method-local code (_build_service, _media_body)
- No module-level google imports in gdrive_api.py

5. _build_service error handling
- Raises RuntimeError (not ImportError) when libs absent
- Error message includes 'storage-gdrive-api' ? actionable for owner
- Credentials sourced from ADC (google.auth.default), NOT config

6. gdrive.py mount backend untouched
- No changes to existing GDriveStorage (mount-only backend)
- Backward compatibility preserved

Test Coverage
- All 39 storage tests pass (30 pre-existing + 9 new gdrive-api tests)
- Config tests: 21/21 pass
- Scheme parsing, registry wiring, hyphen lookup, full round-trip (put/exists/stat/fetch/list/delete) all verified